CHANDIGARH UNIVERSITY

Gharuan, Mohali, Punjab

A Project Report on

QueryLens: A PostgreSQL Query Visualizer

Visualising query execution plans and index performance on a library management database

Submitted in partial fulfilment of the requirements of the course

Advanced Database Management System

Submitted by:

Danish Verma
UID: 24BCY70069
Class: 24AIT-SAP-A

Submitted to:

Mr. Arul
Faculty, Advanced DBMS

Chandigarh University

November 2026

CERTIFICATE

This is to certify that the project report entitled **"QueryLens: A PostgreSQL Query Visualizer"** submitted by **Danish Verma** (UID **24BCY70069**, Class **24AIT-SAP-A**) to Chandigarh University is a record of the project work carried out by the student for the course **Advanced Database Management System**.

The work described in this report was done by the student under my guidance. To the best of my knowledge, it has not been submitted elsewhere for any other course or degree.

Mr. Arul **Date: _____**
(Project Guide)
Chandigarh University

DECLARATION

I, **Danish Verma**, student of class **24AIT-SAP-A** (UID **24BCY70069**), hereby declare that the project report entitled **"QueryLens: A PostgreSQL Query Visualizer"** is my own work, carried out for the course Advanced Database Management System at Chandigarh University. The sources I have used are listed in the References section.

Danish Verma **Signature: _____**
UID: 24BCY70069

ACKNOWLEDGEMENT

I would like to thank my project guide, **Mr. Arul**, for his guidance and encouragement throughout this project. His lectures on query processing, indexing and transaction management gave me the foundation on which QueryLens is built, and his feedback helped me decide what the tool should show.

I am grateful to **Chandigarh University** for providing the learning environment and resources that made this work possible, and to the faculty of the department for their support during the semester.

I also thank the developers of PostgreSQL, Node.js, Express, node-postgres and D3.js. Their open-source software and documentation were essential to this project.

Finally, I thank my family and friends for their constant support and patience.

Danish Verma
UID: 24BCY70069
Class: 24AIT-SAP-A

ABSTRACT

A relational database answers a SQL query by first choosing an **execution plan**: which tables to scan, which indexes to use, in what order to join, and how to sort and group. This choice is made by the query optimizer and is normally invisible to the user. Students learn the theory of query processing, but rarely see what the optimizer actually decides or why one plan is faster than another.

QueryLens is a web-based tool that makes these decisions visible. It is built on **PostgreSQL 18** with a **Node.js / Express** back end and a browser front end that uses **D3.js** for drawing. The tool stores a realistic library management database of seven tables, holding 200,000 loan records, 5,000 members and 2,000 books. Any SQL query typed into the console is executed together with `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`, and the returned plan is drawn as an interactive tree. Node colour shows the share of execution time spent in each operator, link thickness shows the number of rows flowing between operators, and a warning marks operators where the planner's row estimate was wrong by a factor of ten or more.

An **Index Lab** module runs a query, creates an index inside a transaction, runs the query again and rolls the transaction back, so the effect of an index can be measured without changing the database. Seven scenarios cover B-tree, composite, partial, expression and covering indexes, and one case where an index does not help. In testing, indexes reduced execution time by between 1.4× and 722×; a date-range query fell from 151.58 ms to 0.21 ms. The database also demonstrates constraints, a view, a materialized view with a window function, a stored function, and a trigger. Every query run through the tool is recorded in a `query_log` table, and its schema is drawn as a live ER diagram.

The project shows how the theory of query optimization, indexing and transactions applies to a real database system. It also gives a practical tool for understanding and tuning SQL queries.

Keywords: *PostgreSQL, query optimization, execution plan, EXPLAIN ANALYZE, indexing, B-tree, transactions, triggers, materialized views, data visualization*

TABLE OF CONTENTS

CERTIFICATE i
DECLARATION i
ACKNOWLEDGEMENT ii
ABSTRACT iii
TABLE OF CONTENTS iv
Chapter 1: Introduction 1
Chapter 2: Background and Literature Review 2
Chapter 3: Requirements and Tools 4
Chapter 4: System Architecture and Design 5
Chapter 5: Database Design 7
Chapter 6: Advanced Database Features 9
Chapter 7: Implementation 10
Chapter 8: Index Lab and Performance Analysis 11
Chapter 9: Testing and Results 13
Chapter 10: Conclusion and Future Scope 14
References 15

Chapter 1: Introduction

1.1 Overview

Database management systems sit at the centre of almost every software application, from banking and e-commerce to university records. SQL, the language used to talk to relational databases, is *declarative*: the user describes **what** data is wanted, not **how** to fetch it. The job of working out how is left to the database's **query optimizer**, which considers many possible execution strategies and picks the one it estimates to be cheapest.

The quality of that choice can change a query's run time from milliseconds to minutes. A missing index, an outdated statistic or a badly written condition can force the database to read every row of a large table. Understanding execution plans is therefore one of the most important skills in advanced database work, both for developers who write queries and for administrators who tune systems.

PostgreSQL exposes its plans through the `EXPLAIN` command. However, the output is a dense text or JSON document that is hard to read, especially for complex queries with many joins. **QueryLens** turns this output into a picture. It is a complete, working application that stores its data in PostgreSQL and shows, for every query, how PostgreSQL executed it and where the time went.

1.2 Problem Statement

Students of database systems learn about sequential scans, index scans, hash joins and sort operations in theory, but have few ways to observe them in a real system. The raw output of `EXPLAIN ANALYZE` is difficult to interpret, and experimenting with indexes on a shared database is risky, because creating or dropping an index changes the database for everyone.

The problem addressed by this project is therefore: **to build a tool that runs SQL queries on a realistic PostgreSQL database, presents their execution plans visually, and lets the user measure the effect of indexes safely, without permanently changing the database.**

1.3 Objectives

1. Design and normalise a library management database in PostgreSQL with enough data (200,000+ rows) for performance differences to be visible.
2. Use advanced database features: integrity constraints, views, a materialized view, a stored function, a trigger, and transactions with savepoints.
3. Build a web interface where users can type and run any SQL query and see the result as a table and a chart.
4. Draw the execution plan of each query as an interactive tree that highlights expensive operators and bad row estimates.
5. Provide an Index Lab that measures a query before and after adding an index inside a rolled-back transaction.

6. Store every executed query, with its timing and cost, in the database itself, and display the schema as a live ER diagram.

1.4 Scope of the Project

QueryLens is designed as a learning and analysis tool for a single user running PostgreSQL on a local machine. It supports any SQL statement that PostgreSQL accepts. Execution plans are drawn for single `SELECT`, `INSERT`, `UPDATE`, `DELETE` and `WITH` statements, which are the statements PostgreSQL can explain. User accounts, remote access control and multi-database support are outside the scope of this version and are listed as future work in Chapter 10.

Chapter 2: Background and Literature Review

2.1 Query Processing in a Relational DBMS

When PostgreSQL receives a query, it passes through four main stages:

1. **Parsing:** the SQL text is checked for correct syntax and converted into a parse tree.
2. **Rewriting:** rules are applied; for example, a reference to a view is replaced by the view's definition.
3. **Planning / optimization:** the planner generates possible execution plans and estimates the cost of each, then keeps the cheapest.
4. **Execution:** the executor runs the chosen plan as a tree of operators, each pulling rows from its children.

The planning stage is where most performance is won or lost. The idea of cost-based optimization goes back to the System R project at IBM, described by Selinger et al. in 1979, which introduced estimating the cost of access paths from table statistics and choosing join orders by dynamic programming. Modern optimizers, including PostgreSQL's, still follow this approach.

2.2 Cost Estimation

PostgreSQL estimates the cost of a plan in abstract units based on the expected number of disk page reads and CPU operations. Reading one page sequentially costs `seq_page_cost` = 1.0, reading one page randomly costs `random_page_cost` = 4.0, and processing one row costs `cpu_tuple_cost` = 0.01. To estimate how many rows a condition will return (its *selectivity*), the planner uses statistics collected by the `ANALYZE` command: the number of rows in each table, the most common values in each column, and a histogram of value distribution.

When these estimates are wrong, the planner may choose a poor plan. For this reason QueryLens compares the estimated row count (`Plan Rows`) with the actual row count (`Actual Rows`) for every operator and flags any operator where they differ by a factor of ten or more.

2.3 Execution Plan Operators

An execution plan is a tree. Leaf nodes read data from tables; inner nodes combine, filter, sort or aggregate the rows produced by their children. Table 2.1 lists the operators most often seen in QueryLens.

Table 2.1: *Common PostgreSQL plan operators*

Operator	What it does	When it is chosen
Seq Scan	Reads every row of a table and filters them	Small tables, or when many rows match
Index Scan	Follows a B-tree index to matching rows, then reads each row	Few rows match a condition on an indexed column
Index Only Scan	Answers the query from the index alone	All needed columns are stored in the index
Bitmap Heap Scan	Collects matching row locations in a bitmap, then reads pages in order	A moderate number of rows match
Nested Loop	For each outer row, looks up matching inner rows	The outer input is small
Hash Join	Builds a hash table on one input and probes it with the other	Large equality joins
Merge Join	Merges two inputs already sorted on the join key	Both inputs are sorted
Sort / Aggregate	Orders rows; computes count, sum, avg, etc.	ORDER BY, GROUP BY
Gather	Combines rows from parallel worker processes	Large scans split across CPU cores

2.4 The EXPLAIN Command

`EXPLAIN` shows the plan the optimizer chose without running the query. `EXPLAIN ANALYZE` actually executes the query and adds measured values to each node: actual start-up and total time, actual rows and number of loops. The `BUFFERS` option adds how many 8 kB pages were found in memory (shared hits) or read from disk. `FORMAT JSON` returns the plan as a structured document, which QueryLens reads to draw the tree. Because `EXPLAIN ANALYZE` executes the statement, running it on an `INSERT`, `UPDATE` or `DELETE` would change data; Section 6.5 explains how QueryLens prevents this.

2.5 Indexes

An index is a separate data structure that lets the database find rows without reading the whole table. PostgreSQL's default index type is the **B-tree**, a balanced tree that keeps keys in sorted order and finds any key in O(log n) page reads. A B-tree supports equality and range conditions (`=`, `<`, `BETWEEN`) and can also return rows in sorted order, which can remove a separate sort step. PostgreSQL also supports variations that QueryLens demonstrates:

- **Composite index:** an index on several columns, such as `(book_id, loaned_on DESC)`, useful when a query filters on the first column and sorts on the second.

- **Partial index:** an index on only the rows that satisfy a condition, such as open loans `WHERE returned_on IS NULL`. It is smaller and faster to maintain.
- **Expression index:** an index on the result of an expression, such as `lower(email)`, so that case-insensitive searches can use it.
- **Covering index:** an index with extra `INCLUDE` columns, so that an index-only scan can answer the query without visiting the table.

Indexes are not free. Each index takes disk space and must be updated on every insert, update and delete. An index on a column with few distinct values (low selectivity) rarely helps, because the planner prefers a sequential scan when a large fraction of rows matches. QueryLens includes a scenario that shows this.

Chapter 3: Requirements and Tools

3.1 Functional Requirements

Table 3.1: *Functional requirements*

ID	Requirement
FR1	The user can type any SQL statement and run it against the PostgreSQL database.
FR2	Query results are shown as a table and, when a numeric column exists, as a bar chart.
FR3	For explainable statements, the execution plan is drawn as a tree with timing, rows and cost per node.
FR4	Clicking a plan node shows all of its properties and a plain-language explanation.
FR5	The Index Lab compares a query's plan before and after an index, without changing the database.
FR6	The user can create and drop indexes permanently and see how often each index is used.
FR7	The schema is displayed as an ER diagram built from the live database catalog.
FR8	Every query is logged with time, rows, cost and status, and can be reopened from the History view.
FR9	Errors from PostgreSQL are shown to the user with the database's hint, if any.

3.2 Non-Functional Requirements

- **Safety:** analysing a write statement must not apply its changes twice; Index Lab experiments must leave no trace.
- **Performance:** typical queries on the 200,000-row table should return in well under one second; a statement timeout of 20 seconds stops runaway queries.
- **Security of credentials:** database login details are kept in a `.env` file that is excluded from version control.

3.3 Hardware Requirements

A dual-core 2 GHz processor, 4 GB of RAM and 500 MB of free disk space are enough. Development was done on a multi-core x64 laptop with 8 GB of RAM and an SSD, at a screen resolution of 1920 × 1080.

3.4 Software Requirements

Table 3.2: *Software requirements*

Software	Version	Purpose
PostgreSQL	18	Relational database server; stores all data and produces execution plans
Node.js	24 (18+ supported)	JavaScript runtime for the back-end server
Express	4.22	Web framework that serves the UI and the REST API
node-postgres (pg)	8.23	PostgreSQL client library with connection pooling
dotenv	18	Loads database settings from the .env file
D3.js	7.9	Draws the plan tree, charts and ER diagram as SVG
Web browser	Chrome / Edge / Firefox	Runs the user interface
Operating system	Windows 11	Development platform

Chapter 4: System Architecture and Design

4.1 Three-Tier Architecture

QueryLens follows the classic three-tier architecture shown in Figure 4.1. The **presentation tier** is a single-page web application that runs in the browser. The **application tier** is a Node.js server built with Express, which exposes a small REST API. The **data tier** is a PostgreSQL 18 database. The browser never talks to the database directly; every request goes through the API, which controls how queries are run.

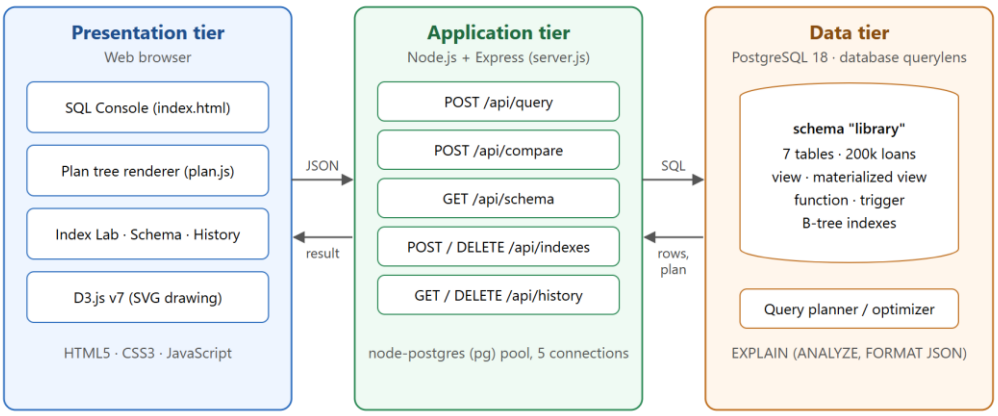


Figure 4.1: *Three-tier architecture of QueryLens*

4.2 Component Description

Presentation tier

The front end is written in plain HTML, CSS and JavaScript without a framework, which keeps it small and easy to follow. It has four views, selected by tabs: **SQL Console**, **Index Lab**, **Schema** and **History**. The file `plan.js` contains the plan-tree renderer, and `app.js` contains the logic for each view. All drawing is done with D3.js, which creates SVG elements that scale cleanly and can respond to clicks and mouse movement.

Application tier

The server (`server.js`) serves the static front-end files and provides the API endpoints listed in Table 4.1. It uses a pool of five PostgreSQL connections. Each connection is opened with `search_path` set to the `library` schema and a `statement_timeout` of 20 seconds.

Table 4.1: *REST API of the application tier*

Method and path	Description
POST /api/query	Runs SQL; returns result sets, execution plan and duration; logs the query
POST /api/compare	Index Lab: plans a query, creates an index, plans again, rolls back
GET /api/schema	Returns tables, columns, keys, foreign keys, row counts, sizes, indexes and other objects
POST /api/indexes	Creates a permanent index on chosen columns and re-analyses the table
DELETE /api/indexes/:name	Drops an index, refusing indexes that back a constraint
GET /api/history	Returns the 100 most recent entries of query_log
DELETE /api/history	Clears the query log

Data tier

PostgreSQL stores the library data in its own schema, `library`, so that it is separate from anything else in the database. Besides the tables, the schema holds a view, a materialized view, two functions and a trigger (Chapter 6). The data tier is also the source of the execution plans: PostgreSQL's planner chooses the plan and its executor measures it.

4.3 Query Execution Flow

When the user clicks **Run**, the server opens a transaction and sets a savepoint. It runs `EXPLAIN ANALYZE` on the statement to get the measured plan, then rolls back to the savepoint, undoing any changes the analysis made. It then executes the statement for real and commits. Finally, it records the query in `query_log`. If any step fails, the whole transaction is rolled back and the error is returned to the browser.

Chapter 5: Database Design

5.1 Requirements of the Library System

The database models a lending library. Books belong to a category and are written by one or more authors. Members borrow books; each loan records when the book was borrowed, when it is due, when it was returned, and any fine charged. The library keeps several copies of each book and must know how many are available at any time.

5.2 Entity-Relationship Diagram

Figure 5.1 is the ER diagram drawn by QueryLens itself from PostgreSQL's system catalog. A key icon marks primary-key columns and a link icon marks foreign-key columns. Each line connects a foreign key to the primary key it references. The header of each table shows its live row count and size on disk.

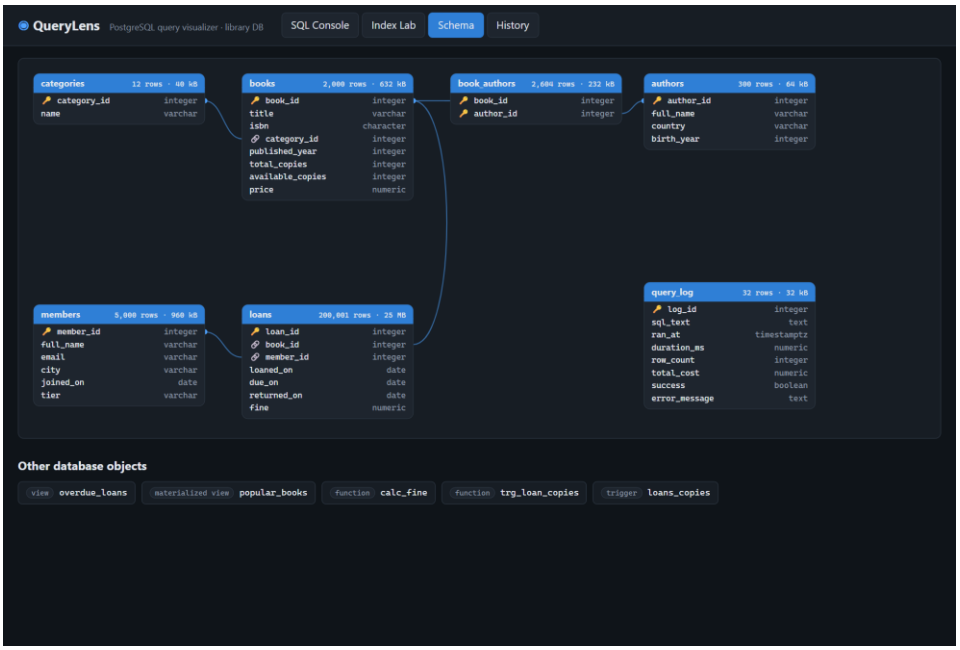


Figure 5.1: *ER diagram of the library schema, generated live by QueryLens*

The relationships are:

- **categories – books:** one-to-many. Each book has exactly one category.
- **books – authors:** many-to-many, resolved by the junction table `book_authors`.
- **members – loans:** one-to-many. A member can have many loans.
- **books – loans:** one-to-many. A book can be borrowed many times.

5.3 Table Definitions

Table 5.1: *Tables of the library schema*

Table	Rows	Key columns	Constraints
categories	12	category_id (PK)	name UNIQUE, NOT NULL

Table	Rows	Key columns	Constraints
authors	300	author_id (PK)	birth_year CHECK BETWEEN 1500 AND 2025
books	2,000	book_id (PK), category_id (FK)	isbn UNIQUE; total_copies, available_copies CHECK ≥ 0
book_authors	2,604	(book_id, author_id) composite PK, both FK	ON DELETE CASCADE
members	5,000	member_id (PK)	email UNIQUE; tier CHECK IN (basic, silver, gold)
loans	200,000	loan_id (PK), book_id (FK), member_id (FK)	fine DEFAULT 0; no secondary indexes (deliberate)
query_log	grows	log_id (PK)	success NOT NULL; ran_at DEFAULT now()

5.4 Normalisation

The schema is in **Third Normal Form (3NF)**:

- **1NF:** every column holds a single atomic value. A book with several authors is not stored as a list in one column; instead each author link is a separate row in `book_authors`.
- **2NF:** every non-key attribute depends on the whole primary key. The only composite key, in `book_authors`, has no non-key attributes.
- **3NF:** no non-key attribute depends on another non-key attribute. Category names are stored once in `categories` rather than repeated in every book row, and member details are stored once in `members` rather than in every loan.

One deliberate exception is `books.available_copies`. It could be computed by counting open loans, but counting 200,000 rows on every lookup would be slow. It is therefore stored, and a trigger (Section 6.4) keeps it consistent. This is a controlled form of denormalisation for performance.

5.5 Data Generation

The seed script (`db/seed.sql`) generates the data inside PostgreSQL using `generate_series` and `random()`, with `setseed(0.42)` so that every run produces the same data. Names, cities and titles are combined from word lists. Loans are spread over the last 1,500 days; recent loans are more likely to be still open, and about 2% of older loans were never returned, so the overdue view has realistic content. After loading, the script calculates fines, corrects `available_copies`, creates the trigger and materialized view, and runs `ANALYZE` so the planner has up-to-date statistics.

Chapter 6: Advanced Database Features

Beyond tables and keys, the project uses several advanced features of PostgreSQL. Each one is visible and testable through the SQL Console's sample queries.

6.1 View: overdue_loans

A view is a stored query that behaves like a virtual table. `overdue_loans` joins three tables and computes how many days each open loan is late, so that users can simply write `SELECT * FROM overdue_loans`. When PostgreSQL plans a query on the view, the rewriter replaces the view name with its definition, and the plan tree shows the underlying joins.

6.2 Materialized View with a Window Function: popular_books

Unlike a normal view, a materialized view stores its result on disk. It is fast to read but must be refreshed with `REFRESH MATERIALIZED VIEW` when the underlying data changes. `popular_books` counts how many times each book has been borrowed and uses the window function `rank() OVER (PARTITION BY ...)` to rank books inside each category. Querying it reads 2,000 stored rows instead of aggregating 200,000 loans, which the plan tree shows as a simple sequential scan.

6.3 Stored Function: calc_fine

`calc_fine(loan_id)` returns the fine owed on a loan: 0.50 per day late, capped at the price of the book. It is declared `STABLE`, which tells the planner that it does not modify the database and returns the same result within one statement. A sample query uses it to list the twenty largest fines currently owed.

6.4 Trigger: loans_copies

The trigger keeps `books.available_copies` correct automatically. When a new open loan is inserted, it decreases the count by one; when a loan is marked as returned, it increases it by one. Together with the `CHECK (available_copies >= 0)` constraint, this means the database itself refuses to lend a book that has no copies left, whichever application tries to do it. When the "Borrow a book" sample is run, the statistics bar in QueryLens shows which triggers fired and how many times.

```
CREATE FUNCTION trg_loan_copies() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' AND NEW.returned_on IS NULL THEN
        UPDATE books SET available_copies = available_copies - 1
        WHERE book_id = NEW.book_id;
    ELSIF TG_OP = 'UPDATE' AND OLD.returned_on IS NULL
                           AND NEW.returned_on IS NOT NULL THEN
        UPDATE books SET available_copies = available_copies + 1
        WHERE book_id = NEW.book_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER loans_copies
```

```
AFTER INSERT OR UPDATE OF returned_on ON loans
FOR EACH ROW EXECUTE FUNCTION trg_loan_copies();
```

6.5 Transactions and Savepoints

Transactions guarantee the ACID properties: a group of statements either completes fully or has no effect. QueryLens relies on this in two places.

Safe analysis of write statements. `EXPLAIN ANALYZE` executes the statement it explains. If the server ran `EXPLAIN ANALYZE INSERT ...` and then the `INSERT` itself, the row would be inserted twice. To prevent this, the analysis runs after a `SAVEPOINT` and is followed by `ROLLBACK TO SAVEPOINT`, which undoes only the analysis. The real statement then runs once and is committed.

Experiments that leave no trace. In PostgreSQL, `CREATE INDEX` is transactional. The Index Lab therefore creates the index inside a transaction, measures the query, and ends with `ROLLBACK`, so the index disappears as if it had never existed. The planner can still use the new index within the same transaction, which is what makes the comparison possible.

Chapter 7: Implementation

7.1 SQL Console

The console is a text area with fourteen sample queries that cover joins, aggregation, window functions, a common table expression, an anti-join, the view, the materialized view, the stored function and the trigger. The user can run the whole text or only a selected part with **Ctrl+Enter**. After a query runs, a statistics bar shows the number of rows, round-trip time, planning time, execution time, estimated cost and buffer usage. Figure 7.1 shows the console after running a query on the `loans` table.

7.2 Plan-Tree Visualization

The renderer in `plan.js` receives the JSON plan and builds the picture in four steps:

1. **Annotation.** The tree is walked from the leaves upward. For each node, the *inclusive* time is `Actual Total Time × Actual Loops`. The node's **own time** is its inclusive time minus the inclusive time of its children. Dividing by the root's time gives the node's share of the whole query.
2. **Estimate check.** The ratio of actual to estimated rows is computed. If it is 10 or more, or 0.1 or less, the node is marked with a warning sign.
3. **Layout.** `d3.hierarchy` and `d3.tree` assign an (x, y) position to each node so that children sit below their parent without overlapping.
4. **Drawing.** Each node becomes a rounded rectangle coloured on a five-step scale from green (under 5% of the time) to red (over 60%). A white bar along its bottom edge is proportional to its share. Links are curved paths whose thickness grows with the square root of the number of rows passing through them.

The most expensive node is selected automatically. Clicking any node shows a side panel with every property PostgreSQL reported, such as filter conditions, join conditions, sort method and buffer counts. The panel also gives a one-sentence explanation of the operator in plain English.

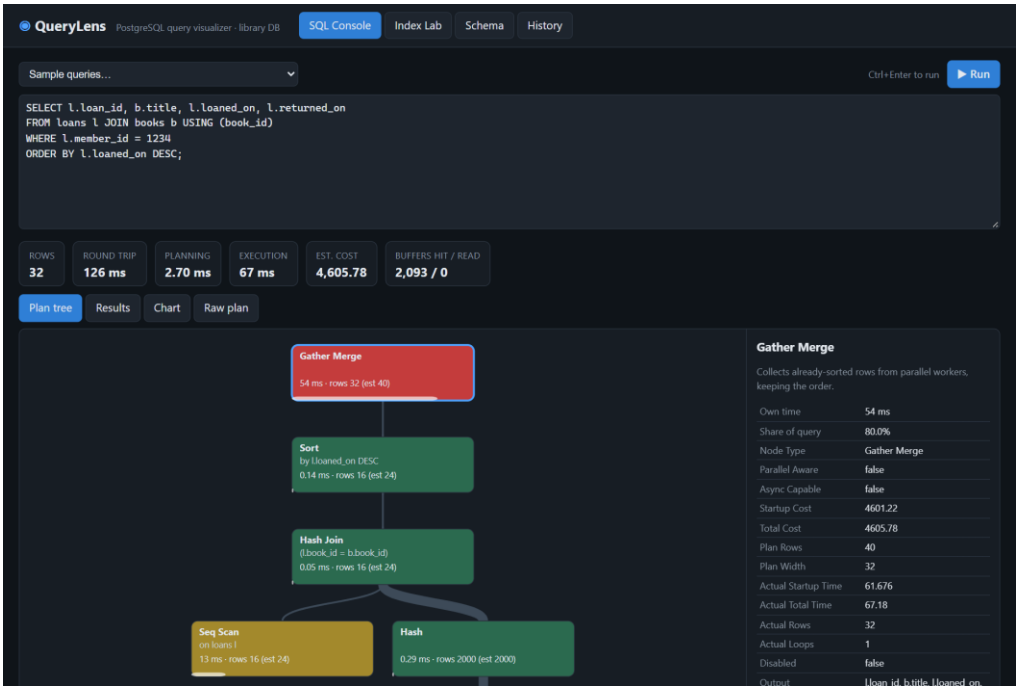


Figure 7.1: *SQL Console showing the plan tree and node details*

7.3 Results and Automatic Charts

Results are shown as a table of up to 500 rows. The **Chart** tab inspects the result: columns whose values are all numeric become value candidates, and the first non-numeric column becomes the label. Rows that share a label are summed, and a bar chart is drawn with D3. The user can change the label and value columns with drop-down lists.

7.4 Query History

After each query, the server inserts one row into `query_log` with the SQL text, duration, row count, estimated cost, and whether it succeeded. The History view charts the round-trip time of the last 50 queries, with failed queries in red, and lists the latest 100. Clicking an entry loads its SQL back into the console.

Chapter 8: Index Lab and Performance Analysis

8.1 How the Index Lab Works

The Index Lab answers a practical question: *would this index make this query faster, and why?* The user chooses a scenario or types a query and a `CREATE INDEX` statement. The server then performs the following steps inside one transaction:

1. Run `EXPLAIN ANALYZE` once to warm the cache, so that both measurements start with the same data in memory.

2. Run `EXPLAIN ANALYZE` again and keep this as the **before** plan.
3. Execute the `CREATE INDEX` statement.
4. Run `EXPLAIN ANALYZE` a third time and keep it as the **after** plan.
5. `ROLLBACK`, removing the index.

The two plans are drawn side by side, with a summary of the speed-up, the scan types used before and after, and bars comparing execution time, estimated cost and buffers touched. Figure 8.1 shows the date-range scenario.

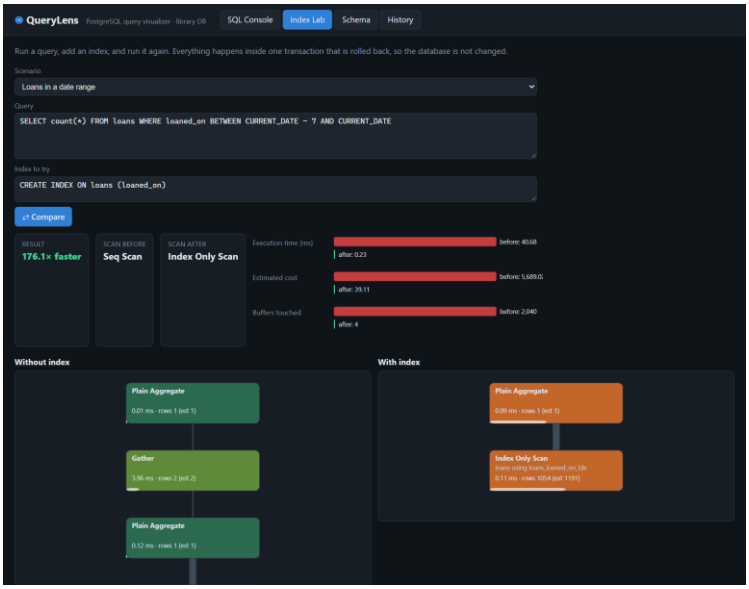


Figure 8.1: *Index Lab comparing a date-range query without and with an index*

8.2 Measured Results

Figure 8.2 shows the execution times measured for all seven scenarios on the development machine (log scale). Exact times vary slightly between runs, but the pattern is consistent.

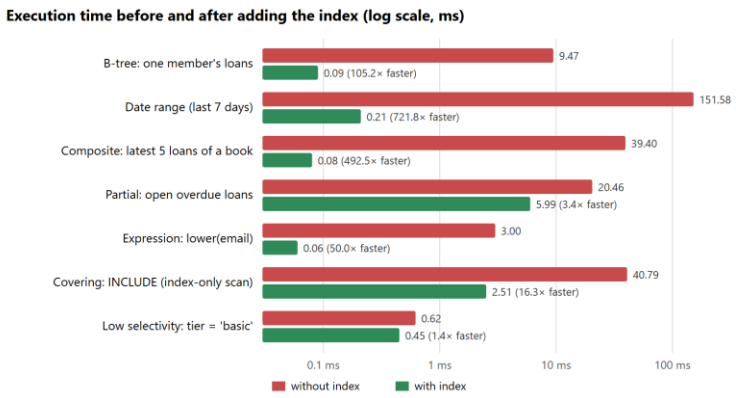


Figure 8.2: *Execution time before and after adding each index (log scale)*

8.3 Analysis

Selective lookups gain the most

The three largest gains come from queries that need very few rows out of 200,000. Without an index, PostgreSQL must read every page of `loans` (2,040 pages, about 16 MB) with a parallel sequential scan. With the index, it walks a B-tree of about three levels and reads only a handful of pages. In the date-range case, buffers touched fell from **2,040 to 4**, and because the count needs only the indexed column, the planner switched to an **Index Only Scan**.

Composite index removes the sort

For the "latest 5 loans of a book" query, the composite index `(book_id, loaned_on DESC)` stores each book's loans already ordered by date. PostgreSQL reads the first five entries for that book and stops: the plan contains a `Limit` over an `Index Scan`, with no `Sort` node at all. This explains the 493× improvement.

Partial and covering indexes

The partial index on open loans holds only about 3% of the rows, so it is small and quick to scan. The gain (3.4×) is smaller because the query still returns several thousand rows that must be read from the table. The covering index lets PostgreSQL count loans per member directly from the index (an Index Only Scan). It is 16× faster even though about 4,000 rows are involved.

When an index does not help

The `tier` column has only three values, and half of all members are "basic". An index on it barely helps (0.62 ms against 0.45 ms), because half the table must still be visited. In general, PostgreSQL keeps a sequential scan when a condition matches a large fraction of the rows. This shows that indexes should be chosen for selective conditions, not added to every column.

Chapter 9: Testing and Results

9.1 Testing Approach

QueryLens was tested at three levels. **Database tests** checked that the setup script builds the schema and loads the expected row counts. **API tests** called every endpoint with a script that ran all fourteen sample queries and all seven Index Lab scenarios and checked that each returned a successful response with a plan. **Interface tests** were done in the browser, checking each view visually and confirming that clicks, tabs and charts behave as expected.

9.2 Test Cases

Table 9.1: *Test cases and results*

#	Test case	Expected result	Result
1	Run setup script on an empty server	Database created; 7 tables loaded with 200,000 loans	Pass

#	Test case	Expected result	Result
2	Run join + GROUP BY sample	12 rows; plan tree with Hash Join and Aggregate	Pass
3	INSERT a loan (trigger sample)	1 row inserted once; trigger fires; available_copies decreases by 1	Pass
4	Run invalid SQL	Error message from PostgreSQL shown; logged as failed	Pass
5	All 7 Index Lab scenarios	Before/after plans returned; index absent afterwards	Pass
6	Create and drop an index from the UI	Index appears in list, then removed; constraint indexes protected	Pass
7	Open Schema view	ER diagram with 7 tables and FK lines; 5 other objects listed	Pass

9.3 Issues Found and Fixed

- **Duplicate ISBNs in generated data.** The first version padded the book number with a repeating "978" pattern, which produced the same ISBN for different books and violated the UNIQUE constraint. The generator was changed to `'978' || lpad(n, 10, '0')`. The constraint caught the bug at load time, before any bad data was stored.

Chapter 10: Conclusion and Future Scope

10.1 Conclusion

QueryLens meets all the objectives set in Chapter 1. It stores a normalised library database of more than 200,000 rows in PostgreSQL. It uses constraints, a view, a materialized view with a window function, a stored function, a trigger, and transactions with savepoints. The web interface runs any SQL query and draws its execution plan as a tree that shows where time is spent and where the planner's estimates were wrong.

The Index Lab turned abstract ideas from the course into measured results. Selective B-tree, composite and expression indexes made queries between 50 and 722 times faster. A composite index removed a sort entirely, and a covering index enabled an index-only scan. The low-selectivity case showed that an index on a column with few distinct values barely helps. Because every experiment runs inside a rolled-back transaction, the database is never changed by accident.

Building the project also showed the value of letting the database enforce its own rules. The UNIQUE constraint caught a data-generation bug at load time, and the trigger with its CHECK constraint guarantees correct book availability, whichever application writes to the table.

10.2 Limitations

- Plans are drawn only for single explainable statements, not for scripts with several statements.

- The tool is designed for one local user and has no authentication, so it should not be exposed to a network.

10.3 Future Scope

- **Plan comparison over time:** store plans in the database and highlight what changed when a query becomes slower.
- **Index recommendations:** suggest indexes automatically from filter and join conditions found in slow plans, using the hypopg extension for hypothetical indexes.
- **Concurrency demonstrations:** show locks, isolation levels and deadlocks by running two sessions side by side.
- **User accounts and roles:** add login and use PostgreSQL roles so that students can only read data, while instructors can change it.

References

1. PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*, Chapter 14: Performance Tips — Using EXPLAIN. https://www.postgresql.org/docs/18/using-explain.html
2. PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*, Chapter 11: Indexes. https://www.postgresql.org/docs/18/indexes.html
3. PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*, Chapter 39: Triggers, and CREATE MATERIALIZED VIEW. https://www.postgresql.org/docs/18/
4. A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
5. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
6. R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill, 2003.
7. P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie and T. G. Price, "Access Path Selection in a Relational Database Management System," *Proc. ACM SIGMOD*, 1979, pp. 23–34.
8. node-postgres documentation. https://node-postgres.com
9. Express documentation. https://expressjs.com
10. M. Bostock, D3.js documentation. https://d3js.org